Java Exceptions: API Design is Hard

Checked exceptions are widely recognized as a poor design decision these days

    Unchecked (aka runtime) exceptions are fine

Predates some important lessons/best practices in API design (field has advanced in the last 30 years!)

Good example of what happens when you aren’t careful about API design

- Hard to use
- Nearly impossible to fix

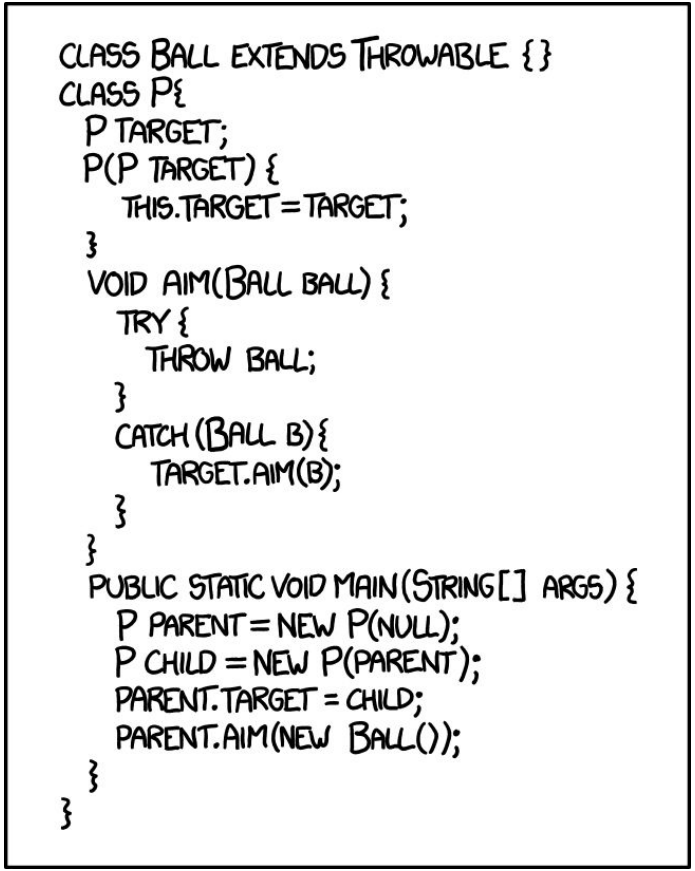
Exceptions (High Level)

When something goes wrong, code will **throw** an exception

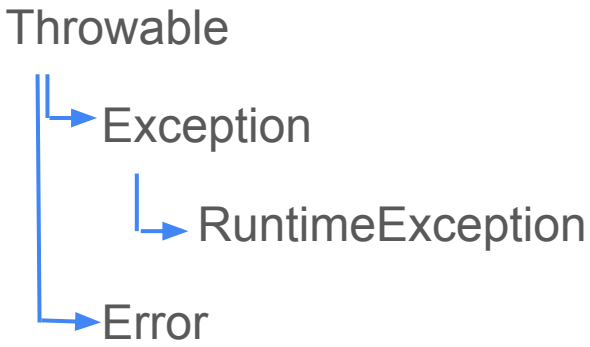
This stops running lines of code and returns up the call stack until it reaches a **catch** block

This block will handle the exception, typically by one or more of:

- Logging the exception
- Re-throwing the exception as a different type or with additional information
- Returning an error code



Types of Exception



Checked Exceptions

Anything that is an Exception but not a RuntimeException is a **checked exception**

- These are a mistake of the past: should either use a return status or generic error handling

Must be **declared** on the method signature

Must be either **caught** or **rethrown** by method callers

Most common in practice: IOException

Common Error Types

AssertionError

OutOfMemoryError (OOM)

Typically for things that shouldn't be specifically caught; indicate exceptionally exceptional circumstances

Finally Block

"try this, possibly handle an error if it goes wrong, and **finally**, do this other thing"

Runs regardless of if an exception was thrown

Throwing an exception in a finally block is really bad:

- masks other errors
- may leave data in a bad state
- very uncharted territory

⇒ use finally cautiously/sparingly

Try With Resources

Alternative to explicit finally block

Avoids resource leak bugs

```
try (FileReader reader = new FileReader("someFile.txt")) {
   reader.readInt();
}
```

Requires that the object implement `Closeable`

When To Throw vs Error Codes

Could a client reasonably handle this error specifically?

⇒ Use an error code (in the past: checked exception)

Would any error handling be of a fairly generic 'something went wrong' type?

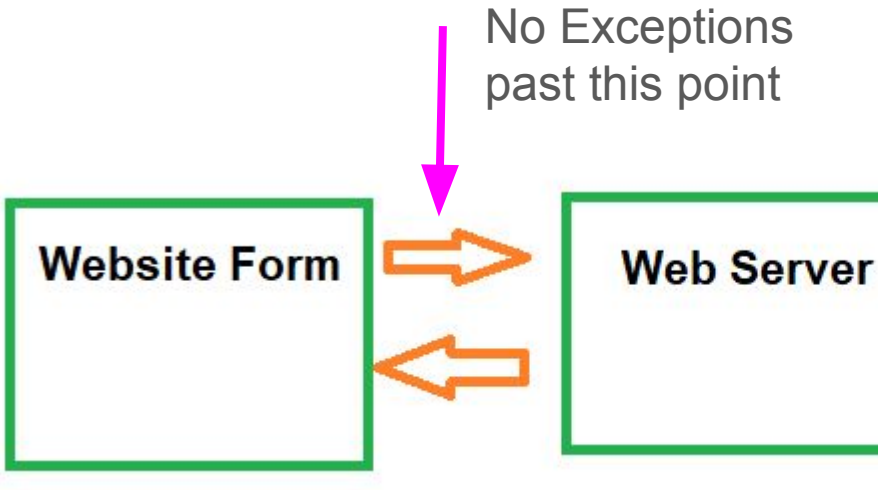
⇒ throw a RuntimeException (typically, IllegalStateException or IllegalArgumentException)

Are errors common enough that all clients should check for them?

⇒ Use an Optional<> return type, possibly with an error code (in the past: return null/-1)

Cross Process/Cross Network Exceptions

Java Exceptions stop propagating at the end of a thread, which includes the edge of a Java Process



Cross Process/Cross Network Exceptions

At the boundary of a process/network call, always include a

```
try {
   // internal server code
} catch (Throwable t) {
   // translate to an error code
}
```

Stack Traces

All Throwables include a **stack trace** - this is automatically populated by the jvm

It starts with what line the exception was thrown at, and continues up the **call stack** with classes and line numbers

Stack Traces

```
3  public class ExceptionExample {
4
5      public static void main(String[] args) {
6          callAMethod();
7      }
8
9      private static void callAMethod() {
10         Integer.parseInt("not a number");
11     }
12 }
```

```
Exception in thread "main" java.lang.NumberFormatException: For input string:
"not a number"
    at java.base/java.lang.NumberFormatException.forInputString(Unknown Source)
    at java.base/java.lang.Integer.parseInt(Unknown Source)
    at java.base/java.lang.Integer.parseInt(Unknown Source)
    at pointerExample.ExceptionExample.callAMethod(ExceptionExample.java:10)
    at pointerExample.ExceptionExample.main(ExceptionExample.java:6)
```

Stack Traces

For **uncaught** exceptions on the **main** thread, they will print to the console and end the program (usually)

For **caught** exceptions, you can do the same with printStackTrace()

In unusual circumstances, you can introspect using getStackTrace()

```
private static void callAMethod() {
    try {
        Integer.parseInt("not a number");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Error Messages

getMessage() is defined on Throwable, but can be and often is **null**

⇒ Use constructors to provide a good message when throwing

⇒ Don't assume the message is helpful when catching

Exception getCause()

"Catch and re-throw" is very common

- Convert checked → unchecked
- Move exceptions across threads
- Sanitize error message

Providing the **original** exception preserves important **debugging** info

Exception getCause()

```java
private static void callAMethod() {
    try {
        Integer.parseInt("not a number");
    } catch (Exception e) {
        throw new IllegalStateException("There is a typo in the code", e);
    }
}
```

```
Exception in thread "main" java.lang.IllegalStateException: There is a typo in the code
    at pointerExample.ExceptionExample.callAMethod(ExceptionExample.java:13)
    at pointerExample.ExceptionExample.main(ExceptionExample.java:6)
Caused by: java.lang.NumberFormatException: For input string: "not a number"
    at java.base/java.lang.NumberFormatException.forInputString(Unknown Source)
    at java.base/java.lang.Integer.parseInt(Unknown Source)
    at java.base/java.lang.Integer.parseInt(Unknown Source)
    at pointerExample.ExceptionExample.callAMethod(ExceptionExample.java:11)
```

Let’s Review

Exceptions:

Three different ways to signal errors to a user: Exception, Error Codes/Return Values, Optional

- What each is and when to use them

try/catch/finally - syntax and what each block does

try-with-resources - when to use it, why to use it

Generics:

- syntax, including ? and ? extends
- How the compiler uses this to enforce type safety
- When should you add a type parameter
- How to use code that has a type parameter